ProveML: Inline Claim Markup for Deterministic Verification of AI-Generated Text

Shane Deconinck
abovebeyond.ai
hello@shanedeconinck.be

August 2026

Abstract

Organizations deploy LLMs in consequential settings (Stanford HAI, 2026; Magesh et al., 2025) and regulation now reaches generated text (European Parliament and Council of the European Union, 2024; European Commission, 2026), yet the text seldom carries a link between a claim and the data behind it that a machine can check (Onweller et al., 2026; Rao et al., 2026). ProveML is a Markdown extension of three inline constructs that declare entities, bind facts to a data source and check qualitative judgments against pre-declared thresholds. Verification is deterministic, exact comparison against a flat key-value store with no model in the loop: milliseconds per response, explainable, usable inside a generation loop or on text the model did not produce. Its semantics is stated formally and its guarantees, that verified means equal to the store and that no unregistered name can decide a judgment, are machine-checked. The verifier also reports coverage, the share of a text's numbers that sit inside any claim.

On a generated educational benchmark and real SEC EDGAR filings, three frontier models of August 2026 (Claude Opus 5, Claude Sonnet 5 and DeepSeek V4 Pro) produce the markup on every query, cover 90–96% of their numbers, and verify 87–100% of claims on the first pass and 92–100% after one correction; stating the binding rule in the specification, a fact may name its own record, lifts the first pass to 95–100%. Given a registry of fourteen named conditions, all three make their qualitative judgments through it, never name a condition it does not hold, and verify 78–100% of those judgments on the first pass and 98–100% after one correction; the failures are the word the question invited, refused by the bound. The specification, verifier, benchmarks and run artifacts are published.

Keywords: AI verification, markup language, deterministic verification, structured data, threshold registry, hallucination detection, inline tagging

1 Introduction

Large Language Models (LLMs) produce fluent, contextually appropriate text that untrained readers can no longer reliably distinguish from human-authored content (Clark et al., 2021; Jakesch et al., 2023; Jones and Bergen, 2026); readers who write with these models daily still can (Russell et al., 2025). The fluency carries no reliable signal of the model's own uncertainty: deployed models rarely volunteer uncertainty even when they are wrong, verbalised confidence is systematically overconfident, and the training pipeline rewards a confident guess over an admission of ignorance (Zhou et al., 2024; Xiong et al., 2024; Kalai et al., 2025). And the fluency coexists with a fundamental reliability problem: **LLMs hallucinate** (Ji et al., 2023). They generate text that is plausible but factually incorrect: fabricated citations, 3–13% of the citation URLs supplied by commercial models and deep research agents in 2026 having no

archived record and likely never having existed (Rao et al., 2026); numbers with no basis in the source (Cao et al., 2024); and statements whose citations do not support them (Liu et al., 2023).

Hallucination is structural, not incidental: calibrated language models must hallucinate at a rate approaching the fraction of facts appearing exactly once in training (Kalai and Vempala, 2024), and standard training pipelines reward guessing over acknowledging uncertainty (Kalai et al., 2025). Regulation no longer adds urgency; it adds exposure. The EU AI Act's transparency duties have applied since 2 August 2026, with the Commission's Article 50 guidelines adopted two weeks before; the Digital Omnibus deferred only one of those duties, machine-readable marking by systems already on the market, to 2 December 2026 (European Parliament and Council of the European Union, 2024; European Commission, 2026; European Parliament and Council of the European Union, 2026). But the Code of Practice that operationalises these duties concerns marking the *origin* of content and says nothing about its accuracy (European AI Office, 2026): a text can be fully compliant and wrong. And the problem has not gone away with better models: the leading legal research tools hallucinated on 17–33% of queries when tested in 2024, despite vendor claims of near-elimination (Magesh et al., 2025), and a 2026 survey of court filings found more than a thousand containing fabricated citations, a number growing year over year (Liu et al., 2026).

The models are meanwhile in production. Generative AI is used in at least one business function at 70% of surveyed organizations (Stanford HAI, 2026), in legal research (Magesh et al., 2025) and in clinical documentation (Wright et al., 2026). Even systems built to cite fall short: in 2026 the citations of frontier deep-research agents checked out against their sources for only 39–77% of citations, and 10.7% of the citation URLs of the two commercial deep research agents had no archived record and likely never existed, against 4.8% for search-augmented models (Onweller et al., 2026; Rao et al., 2026), three years after the first generative search engines fully supported about half of their sentences (Liu et al., 2023), a rate the ALCE benchmark reproduced for the best models of the time (Gao et al., 2023); and whether a citation holds is judged by a person or by another model rather than by a deterministic check (Rashkin et al., 2023; Onweller et al., 2026).

The response this paper takes is not to make the model more truthful but to make its claims *checkable*: every marked claim either matches an addressable record or is flagged, deterministically, before anyone reads it. In short, **ProveML is iXBRL for AI-generated text.** Financial reporting solved a version of this problem two decades ago by embedding machine-readable tags in human-readable filings, so a regulator can audit a number without reading the prose around it (XBRL International, 2013). ProveML applies that idea where the author is a model rather than a person, and adds what that setting needs: entity scoping, so a value is bound to the record it describes, and a threshold registry, so a qualitative judgment is a declared predicate rather than a choice of adjective; Figure 1 shows what that buys the reader.

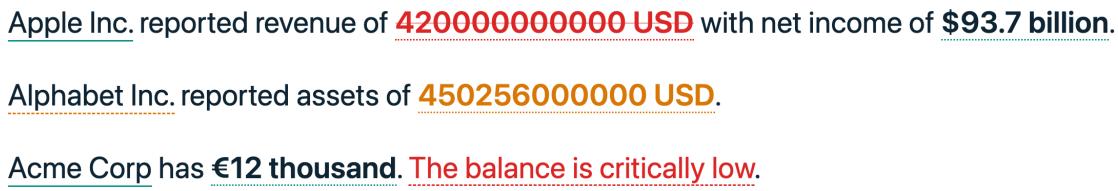

Figure 1: What the reader sees after verification: a wrong revenue figure struck through, a claim about a company the store does not know left open, a judgment whose condition does not hold marked as failed. The model wrote 93736000000; the store's display rule turns it into "$93.7 billion" once it has verified. The wrong revenue figure never verified, so it stays as the model wrote it.

Two things decide whether ProveML fits a use. First, verification is exact: the model writes the value as the store holds it, `391035000000`, and "$391 billion" in its place fails. Readability is the renderer's job, not the model's: the store declares a display rule for the field and the renderer applies it after verification (Section 2). That is the guarantee, and it asks for a store that holds canonical values. Second, the threshold registry is the deployment's: the names a model may use, and the bound behind each, are declared before generation, so a judgment the deployment never named cannot be made checkable by the model. Section 3 measures what models do with such a registry. Section 6 gives the full list of limits.

This paper is deliberately compact: it gives the idea, the three constructs, what the mechanism costs, and what we measured on three models that were frontier in August 2026 (Claude Opus 5, Claude Sonnet 5 and DeepSeek V4 Pro). The complete specification (verification algorithm, comparison semantics, rendering) is in the accompanying technical report, together with the July 2026 study of smaller models that preceded this one, whose results this paper does not reproduce; the three-valued condition semantics is stated in Appendix B. Both documents are published in the artifact repository alongside every benchmark, every run the results draw on, and the scripts that regenerate every results table.[1]

2 ProveML

ProveML extends Markdown with three constructs, using characters (`@`, `%`, `?`) that do not conflict with standard Markdown syntax. Instruction-tuned models routinely answer in Markdown (Sun et al., 2025), and a CommonMark-conformant renderer passes the constructs through unchanged, because characters not given an interpretation by any Markdown rule are parsed as plain textual content (MacFarlane, 2024); in our runs every model produced the markup on every query.

```
(* Simple form: entity, then facts follow *)
@[sensor:42]{Sensor X} reads %[value]{29.99} with %[stock]{150} in stock.

(* Scoped form: facts inside entity braces *)
@[sensor:42 "Sensor X"]{reads %[value]{29.99}, %[stock]{150} in stock}

(* Nested scopes: outer entity with multiple inner entities *)
@[facility:7 "Plant North"]{hosts
  @[sensor:42 "Sensor X"]{reading %[value]{29.99}} and
  @[sensor:43 "Sensor Y"]{reading %[value]{18.5}},
  with %[sensorCount]{12} sensors total}
```

Entity references `@[type:id]{name}` declare the subject. The verifier checks the display name against the store (`sensor:42.name`), and the entity becomes the context for the facts that follow. Binding is structural: which entity a fact binds to follows from the markup by fixed rules, not from a model's reading of the sentence. A fact inside scoped braces binds to that entity (lexical scope); a fact outside binds to the last simple-form entity at the current depth (linear carry-forward), and closing a scope restores the context that was in force when it opened.

Fact references `%[field]{value}` claim a value. The verifier checks `store`$[\textit{entity.field}] = \textit{value}$ by exact string equality on the store's canonical representation (the store follows below; where the store declares a unit, the claim carries it, as in `%[balance]{-12400 EUR}`). Four outcomes: *verified*, *mismatch*, *unverifiable* (no such field), *no context*. A fact may also name its

[1] https://github.com/abovebeyond-ai/proveml-research — reference implementation: https://github.com/abovebeyond-ai/proveml (npm: `proveml`).

own record, `%[student:20414.passRate]{53}`, for a sentence whose subject is not the nearest entity; Section 3 shows why that is needed.

Inference references `?[label: CONDITION]{text}` make a qualitative judgment checkable. The condition names a threshold from a registry defined outside generation; the model cannot invent a comparison value, a bound, or a direction, and an unregistered name is an error rather than a claim. Conditions compose with `AND`, `OR`, `NOT` and can reference earlier labels:

```
?[low: IS_LOW]{critically low score}
?[missing: IS_MISSING]{no recent data}
?[risk: @low AND @missing]{low score with missing data}
```

The fact store is a flat key-value index in the Entity-Attribute-Value pattern (Nadkarni et al., 1999; Dinu and Nadkarni, 2007); any source with records and fields flattens into it as `type:id.field` → `value`, with optional companion keys for units and nested sub-paths for hierarchical data:

```
account:901.holder                        -> "Acme Corp"
account:901.balance                       -> -12400
account:901.balance._unit                 -> "EUR"
account:901.overdue                       -> 3
account:901.transaction:2026-01.amount    -> 5200
account:901.transaction:2026-01.amount._unit -> "EUR"
```

The store is the single point of trust and the canonicalization point: the guarantee is "consistent with the fact store", not "true", and verification is always relative to an immutable store state (deployments can bind a snapshot identifier to each result). Arithmetic lives in the data layer, not the verifier: a cross-entity difference is materialized as an addressable fact (`region:EU._salesDiff`) and bounded by a registered predicate, so every operand of every comparison is itself auditable. So does presentation: a companion key `revenue._display = currency:USD:1` tells the renderer to show a verified `391035000000 USD` as "$391.0 billion", with the canonical value kept on the element and in the audit view. The claim, the check and the stored text stay exact; only what the eye meets is rounded, by a rule the model never sees (Figure 2). Tolerance in the verifier would have bought the same readability at the price of the guarantee.

The threshold registry follows a pattern shared by clinical reference intervals, JSON Schema validation and monitoring alert rules: a bounded condition on a single field is given a name and a human-readable label, so the judgment can be invoked and audited by that name rather than restated as an expression wherever it is used (Ozarda, 2016; Wright et al., 2022; Prometheus Authors, 2026). Each definition has a name, a field, one predicate from a deliberately small vocabulary (Table 1), an optional unit constraint, a label and a source for audit:

```
IS_LOW:        score lt 25         "critically low"
IS_MODERATE:   score between 25 50 "moderate"
IS_ACTIVE:     status in {A,B,C}   "active"
IS_MISSING:    value is_null       "no data"
MUCH_HIGHER:   _scoreDiff diff_gt 20 "much higher"
IS_NEGATIVE:   balance lt 0 [EUR]  "negative balance"
```

Verification resolves each construct against the store — string equality for entities and facts, typed comparison for thresholds — with no model in the loop, so it is exact, explainable and

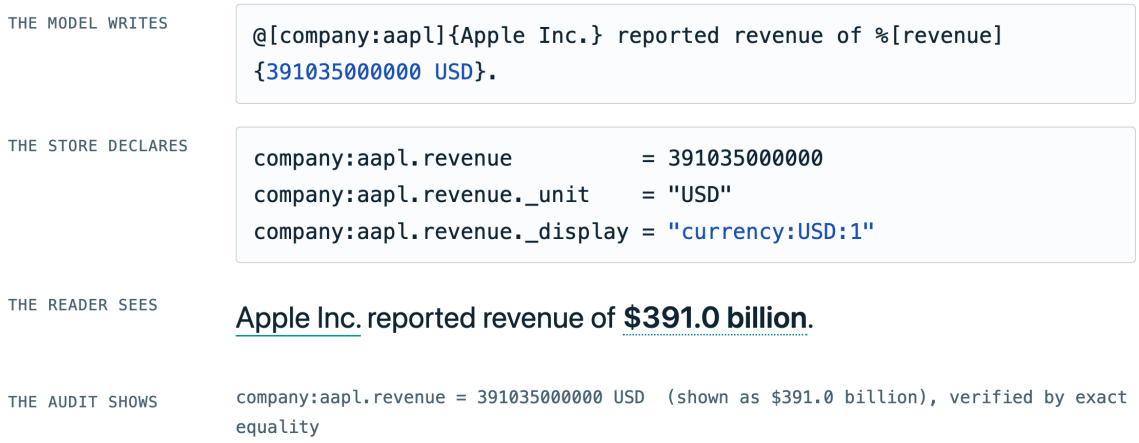

Figure 2: One claim, three views. The model writes the canonical value; the store declares the unit and a display rule next to the field; the reader sees the rounded form; the audit view keeps the exact value and says how it was shown. The model never rounds, so it can never round wrong.

Operator	Display label	Example
Core (numeric comparison)		
`lt`	is less than	`price lt 10`
`lte`	is at most	`rating lte 3`
`gt`	is greater than	`stock gt 0`
`gte`	is at least	`score gte 75`
`eq`	equals	`status eq "active"`
`neq`	is not	`status neq "archived"`
`between`	is between	`score between 25 50`
Extended		
`in`	is one of	`category in {A,B,C}`
`is_null`	is missing	`value is_null`
`diff_gt`	exceeds by more than	`_scoreDiff diff_gt 20`

Table 1: Threshold operator vocabulary. Core operators cover ordering, equality and ranges. Extended operators handle set membership, missing values, and cross-entity comparison.

effectively free (Section 4). A judgment the verifier cannot evaluate, because the threshold it names was never registered or the value it needs is missing, is reported as unverifiable, and negating it does not make it verified: `NOT` of an unverifiable condition is still unverifiable. Without that rule a model could get a claim past the verifier by negating a threshold that does not exist. Because errors come back as specific messages (“`%[passRate]{7} in student:42: should be 5`”), verification can sit inside a generation loop that feeds each error back to the model; and because the verifier only reads the text, it can equally audit text it did not generate. A verification rate counts only what is inside markup, so a text that marks one number and leaves nine in prose would score 100%; the verifier therefore also reports *coverage*, the share of standalone numbers that sit inside a claim (years, list markers and code excluded), and in strict mode treats each number outside one as a finding. Figure 3 shows the pipeline; Figure 4 the audit rendering, where each claim carries its proof path. The complete grammar is in Appendix C; Appendix B states the semantics and the properties the mechanised model proves for it.

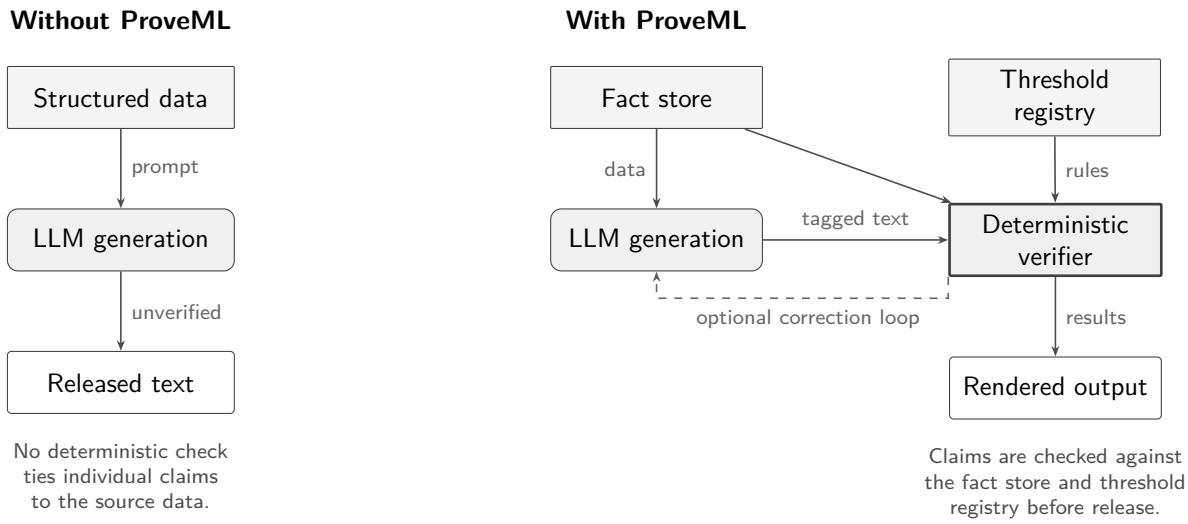

Figure 3: Without ProveML (left): data goes in, text comes out, no verification. With ProveML (right): every claim is checked against the fact store and threshold registry, with an optional correction loop. Rounded boxes denote generative components; squared boxes denote data stores and deterministic processing.

Apple Inc. [company:aapl.name] reported revenue of **$391.0 billion** [company:aapl.revenue] with net income of **$93.7 billion** [company:aapl.netIncome] and earnings per share of **6.08 USD/shares** [company:aapl.eps].

Microsoft Corporation [company:msft.name] reported revenue of **$245.1 billion** [company:msft.revenue] with total assets of **$512.2 billion** [company:msft.assets] and a debt-to-equity ratio of **0.16** [company:msft.debtToEquity].

Figure 4: Audit mode: the fact-store path checked for each claim is visible inline (`company:aapl.revenue`, `company:msft.debtToEquity`). A reader, or an auditor months later, can take any number back to its record without a model.

3 What We Measured

Setup. Three models that were frontier at the time of writing (August 2026): Claude Opus 5 and Claude Sonnet 5 (Anthropic, through the Claude Code CLI with default settings), and DeepSeek V4 Pro (`DeepSeek-V4-Pro-0813`, open weights under MIT, served by Together AI). Two benchmarks: 28 English prompts over a generated educational dataset of 741 pupils in 95 class offerings,[2] and 10 English prompts over real SEC EDGAR FY2025 filings (2 entities, 11 fields). Three runs per model and benchmark, one correction pass: the verifier's errors are fed back once, and the corrected answer is accepted only if it verifies better without dropping claims. Verification rate is a per-query macro-average; a response with no markup scores 0%, an empty answer scores 0% rather than being dropped. Context is sliced per prompt from

[2]Generated, not de-identified: every record is drawn from a latent-ability model, the school is fictional, and the seeded generator ships with the artifacts. We state this from experience: an earlier draft used a de-identified extract of real records under the label "synthetic"; it was withdrawn, replaced, and every educational result re-measured on the replacement.

the benchmark's own entity lists — an oracle retriever, which makes the rates an upper bound conditional on retrieval. The system prompt asks for entity and fact markup; no run produced an inference construct, so all rates measure those two. Every number in this section regenerates from the published runs (`experiments/frontier-summary.mjs`, `frontier-residuals.mjs`); the verifier's own conformance test (20 planted errors, all caught) is in Appendix A.

Benchmark	Model	First pass	One correction	Fully verified	Coverage
Education	Claude Opus 5	86.5% ± 0.3	94.8% ± 4.8	23.3/28	91.4%
	Claude Sonnet 5	89.3% ± 2.1	92.2% ± 3.6	21.3/28	92.3%
	DeepSeek V4 Pro	97.2% ± 1.3	99.5% ± 0.8	27.7/28	94.2%
Finance	Claude Opus 5	97.9% ± 0.6	100% ± 0.0	10/10	95.8%
	Claude Sonnet 5	96.5% ± 1.2	96.5% ± 1.2	7/10	89.8%
	DeepSeek V4 Pro	100% ± 0.0	100% ± 0.0	10/10	96.4%

Table 2: Three frontier models, three runs each (mean ± sample sd over runs). First pass and after one correction: per-query verification rate. Fully verified: queries on which every claim verified after at most one correction (mean over runs). Coverage: numbers inside a claim as a share of all standalone numbers in the delivered text, pooled over runs, by the verifier's own definition.

Finding 1: the mechanism is within reach of current models. No model, on any of the 342 query-runs, produced an empty answer or an answer without markup, and no call failed. First-pass verification runs from 86.5% to 100%, coverage from 89.8% to 96.4%, and one correction pass lifts Opus 5 by eight points on education and closes finance entirely (Table 2). The open-weight model is the strongest of the three on both benchmarks, at a third of the latency (3.4 s per query against 7.9 and 11.8 s, Section 4). Rate and coverage move together here, unlike in the earlier study of small models, where they diverged by more than thirty points at identical rates (technical report): at the frontier, a model that marks up reliably also marks up nearly everything.

Finding 2: what remains is mostly the binding rule, not the model. Of the 157 errors still standing after the correction pass on the educational benchmark, 108 (69%) have one shape. The sentence names the pupil and then the class — *"@[student:20414]{Amir Janssens} of @[offering:10056]{5OL} has a pass rate of %[passRate]{53}%"* — and linear carry-forward binds the pupil's facts to the class, the nearest preceding entity. The data is right, the English is right, and the verifier reports `offering:10056.passRate` not found. It touched 17 of the 35 non-converged query-runs, and the correction pass could not repair it because nothing in the error message says which entity the fact should have gone to. The scoped form exists for exactly this, and no model used it. We have therefore made a fact able to name its own record, `%[student:20414.passRate]{53}`, which the grammar's `path_field` production always admitted and the verifier now honours, and then measured it (Finding 3). Wrong values are 40 of the 157 (25%), almost all on aggregation prompts that ask for a rank or a count the store does not hold. Two errors are a bound written as a fact (`%[passRate]{60}` before any entity, for "a 60% threshold"): a number that is not a record, which is what the inference construct is for.

Finding 3: told the rule, the models follow it, and the residue goes. We re-ran the study with the binding rule, one domain rule and one example line added to the system prompt, and nothing else changed (Table 3). On finance all three models verify every claim on the first pass in every run. On education Sonnet 5 rises from 89.3% to 99.4%, DeepSeek

Benchmark	Model	First pass	One correction	Fully verified	Coverage
Education	Claude Opus 5	95.4% ± 7.5	97.1% ± 4.7	25.7/28	97.9%
	Claude Sonnet 5	99.4% ± 0.2	99.4% ± 0.2	25.7/28	91.1%
	DeepSeek V4 Pro	99.2% ± 1.5	100% ± 0.0	28/28	95.1%
Finance	Claude Opus 5	100% ± 0.0	100% ± 0.0	10/10	98.2%
	Claude Sonnet 5	100% ± 0.0	100% ± 0.0	10/10	94.4%
	DeepSeek V4 Pro	100% ± 0.0	100% ± 0.0	10/10	97.6%

Table 3: The same study under the second prompt, which adds to the first the binding rule (a fact may name its own record when the sentence names two subjects), one domain rule (education: a cutoff from the question is not a fact; finance: no computed differences, ratios or totals) and one example line showing the explicit-record form. Everything else — models, benchmarks, slices, runs, correction budget — is identical to Table 2.

from 97.2% to 99.2% and to 100% after one correction, and Opus 5 from 86.5% to 95.4%: two of its three runs verify 100% of claims on the first pass, and in the third it reverted, on three pupil queries, to the very phrasing the rule addresses ("Ali Peeters of 6WEWI has a pass rate of ...") without the explicit record, which accounts for the spread. The 108 binding errors of the first study are 16 in the second, all in that one run. What remains elsewhere is small and mixed: a handful of wrong aggregates, and a model asking for a field the store does not hold (`passRateDiff`, `unevaluatedCount`) rather than inventing a number for it, which is the failure the design prefers. The rules now ship with the implementation as a generated system prompt (`proveml/prompt`), built from the store and the registry so the names the model sees are the names the verifier resolves; each rule in it is tied to a measured gap, and this is how we expect the prompt to evolve.

A first pass of this study measured the harness. The context handed to the models named the fields `students` and `rate` where the store held `studentCount` and `passRate`. Every model reproduced what it was shown, and those two names were the two most frequent "field not found" errors of that pass. We record it because the same mismatch was present in the July 2026 study, whose residual-error analysis it inflated, and because it is a general lesson for deployments: the vocabulary the model sees must be the vocabulary the store resolves, or addressability errors will be measured that no model made.

The third construct: judgments. The two studies above asked for entities and facts. A third study, run in September 2026, asks for the judgment construct. Twenty questions over the education store invite a qualitative assessment and name no cutoff ("How is 6VZ doing?", "Is 6ZW a strong class?", "Why is 6VZ doing so badly?", "Is 3BS's teacher effective?"): eight that the data supports, four at a registered bound (6ZW at a pass rate of 74 against `IS_STRONG` at 75 or more; a pupil with 29 absences against `IS_GREY_RISK` above 30), four that presuppose a judgment the data refutes, and four that invite a judgment no registry entry covers. The registry is the education part of the example registry that ships with the package, fourteen names selected by field, and the prompt is the one `proveml/prompt` generates from the store and that registry. The same questions ran under the facts-only prompt of the second study as the baseline. Same three models, three runs each, one correction pass. Which registered conditions hold for each record in a question's context is computed by the verifier and saved with the run, so the boundary and contrary questions score themselves.

Finding 4: models use the construct, never invent a name, and reach for the word the sentence wants. With the registry every Opus 5 response, 19 of 20 Sonnet 5 responses

Model	Responses	Judgments	First pass	One correction	Boundary, contrary
Claude Opus 5	20/20	67.7 ± 5.5	92.6% ± 1.1	100% ± 0.0	7.3 → 8.0 of 8
Claude Sonnet 5	19.3/20	42.3 ± 4.2	77.8% ± 3.3	98.2% ± 1.5	3.7 → 7.7 of 8
DeepSeek V4 Pro	18.0/20	41.3 ± 1.2	100% ± 0.0	100% ± 0.0	8.0 → 8.0 of 8
Without a registry	0/20	0	18 to 26 uncheckable qualitative words per run		

Table 4: The judgment study: 20 questions that invite a qualitative assessment, mean ± sample sd over three runs. Responses: those carrying at least one judgment. Judgments: emitted per run. First pass and one correction: the share of emitted judgments whose registered condition holds. Boundary, contrary: of the eight such questions, those on which no judgment the data refutes survived, before and after the correction. No model in any run named a threshold the registry does not hold.

and 18 of 20 DeepSeek responses carry at least one judgment: 454 judgments over nine runs, 411 of them verified on the first pass and 425 of 427 after one correction (Table 4). Without it, the same questions draw 18 to 26 qualitative words per run (strong, low, at risk, average, concern) that no verifier can see; that is the baseline this construct replaces. Not one of the 454 named a threshold outside the registry. On the four questions with no matching entry the models stated the numbers and said the data holds no such field, or used a registered judgment that holds; one correction on Sonnet 5 produced a malformed label, the single judgment left unverified. The 33 first-pass failures are almost all one thing: "small sample" at 5 or 7 pupils against a bound of 5, "strong" at 74 or 63 against 75, "reliable sample" at 7 or 8 against 10, "risk of grey marks" at 29 absences against 30. The model reached for the word the question invited, the registry's bound said no, and the correction removed the claim rather than the number. That is the overreach the registry exists to catch, and it is caught deterministically: on the eight boundary and contrary questions Sonnet 5 answered 3.7 of 8 soundly on the first pass and 7.7 after one correction, Opus 5 went from 7.3 to 8, DeepSeek was sound on all 8 in both. A second observation from the setup rather than the runs: an `is_null` judgment is vacuously true on a record type that never carries the field, so `IS_MISSING` on `evaluated` would have verified for every class; it was excluded, and the specification should say that a missing field is not a null value.

4 Deployment

The evaluation measures whether models can produce verifiable markup. This section reports what it costs to run and what a deployment has to build, because both are load-bearing for anyone deciding whether to adopt the mechanism, and neither follows from the tables above.

Cost of verification: negligible next to generation. Verifying the 2,185 claims in the 252 stored final responses of the educational runs against the 4,826-key fact store takes 0.66 s on a laptop, mean of 20 passes — 2.6 ms per response, 0.30 ms per claim (the script `deployment-numbers.mjs` in `experiments/`, run with `-tag frontier`, recomputes every figure in this section). The cost per claim grows with the store, because the verifier scans the store for each fact to establish that its subject is unique; an indexed store would bring it to microseconds, and an earlier version of this paper printed a microsecond figure that only holds on a toy store. Against generation latency (3.4 to 11.8 s per query, summed over the correction pass) this is free, which is what allows verification to sit inside the generation loop rather than after it.

Cost of generation: markup and retries. Two overheads are real. Marked-up responses ran 49% (Opus 5), 66% (Sonnet 5) and 79% (DeepSeek) longer in characters than the same

text with the markup stripped; the shortest answers pay the highest relative price, because the markup is a fixed cost per claim. That is a direct output-token cost. The correction pass adds a second: 77% of query-runs needed none, and the mean is 1.23 generations per query (1.06 for DeepSeek, 1.33 for Opus 5). A deployment that allows one pass and no more captures what we measured; the July study found that further passes rarely change the outcome.

What a deployment has to build. The fact store is the work. Flattening records into `type:id.field` is mechanical once the entities are decided, but deciding them is a modelling exercise: a normalized database with joins does not announce what counts as one addressable entity, and that choice determines which claims can be bound at all. Derived values are the recurring case — a count, a difference, a ratio the report computes on the fly — and the rule is that arithmetic belongs in the data layer, materialized as an addressable fact, rather than in the verifier (Section 2). A threshold registry is optional at the entity-and-fact level and small when present: the example registry that ships with the implementation has twenty-one entries, and neither benchmark needed one (no run produced an inference), so its cost falls entirely on the deployments that use it. The vocabulary the model is shown must be the store's vocabulary (Section 3); that is a one-line rule and an easy one to break.

The retrieval assumption, stated plainly. Our headline rates assume retrieval has already succeeded: context slices come from the benchmark's own entity lists, an oracle retriever. The July study's full-context ablation shows what happens without slicing for small models — zero constructs on every query, with the numbers emitted as plain prose instead (technical report); we did not repeat it at the frontier. Context selection is a precondition of the mechanism, and its quality in a real deployment is the largest variable this paper does not measure. Treat the reported rates as an upper bound conditional on retrieval, and budget for the retrieval work accordingly.

Minimal integration. The reference implementation is an npm package whose verifier has no runtime dependencies (only the optional markdown-it plugin needs a peer):

```
import { verifyProveml } from 'proveml/verify';

const store = { 'student:42.name': 'Alex', 'student:42.passRate': 5 };
const text  = '@[student:42]{Alex} scored %[passRate]{5}%.';

const { verified, total, errors } = verifyProveml(text, store);
// -> 2 / 2, no errors
```

A deployment adds three things to an existing LLM call: the store, a system prompt that asks for the markup, and the verifier on the response. The verify-fix loop and the rendering layer are optional on top.

The store's own provenance. The first limitation in Section 6 — verified means consistent with the store, not true — marks the one link the verifier cannot close, and the implementation gives a deployment a discipline for it rather than leaving it to trust. In the review layer (`proveml/review`), every store value carries its evidence: a quote checked verbatim against an archived snapshot of its source, a declared derivation, or a declared absence, because one cannot quote a source not having something. The machine re-checks those links at build time and refuses to emit the review page when any fails; the one link it cannot check — whether a value is a fair reading of its evidence — is judged by a person on that page, and each judgement is saved under a hash of exactly what it judged, so it expires the moment the evidence changes and

review shrinks to the diff. The gate is awaitable in a pipeline (`proveml review -await` exits nonzero while readings are unjudged or flagged), and how a sign-off is attested is an adapter, so a deployment can bind it to a name, a key signature, or a ledger anchor. This paper applies the layer to itself, under a versioned review protocol:[3] every number is bound to the file that regenerates it, every citation to a verbatim passage in an archived copy, every unsourced statement is flagged by a model pass and resolved against the repository, and what remains is judged by a named person on the review page. The round of 11 September 2026 went over 229 readings that asked for a judgment and left one false statement and twelve rewordings behind it; the next pass recorded nothing. The protocol's own limits are stated in it: this first round was resolved by the author and agents the author ran, and an independent reviewer has not yet judged a reading.

5 Related Work

The idea of tagging human-readable text for machine resolution is old: RDFa binds spans of prose to entities in a structured vocabulary and assumes the author meant it (W3C, 2015); iXBRL embeds machine-readable tags in financial reports for automated audit (XBRL International, 2013). ProveML adds the verdict to that lineage — not what a span refers to, but whether the claim survives comparison with the record. Among recent systems, Proof-Carrying Numbers (Solatorio, 2025) is the nearest neighbor (claim-bound numeric tokens, deterministically verified in the renderer, with declared tolerance policies; its paper describes neither entity scoping in prose nor a named vocabulary for qualitative judgments), and SymGen (Torroba Hennigen et al., 2024) takes the opposite strategy: the model emits references and a parser substitutes the values, so a wrong number is impossible and so is reporting one; the technical report compares the two on identical runs. Claim-locked reporting (Fan et al., 2026) is the 2026 form of that strategy — the numbers, their direction and the permitted strength of language are fixed before the model writes connective prose — and shares ProveML's premise that qualitative wording must be tied to a declared threshold, while giving up the ability to audit text it did not produce. VeriFin (Hall et al., 2026) checks financial claims against XBRL facts with an SMT solver, placing the arithmetic in the verifier where ProveML places it in the data layer. Yang et al. (2026) measure the failure class our residual errors belong to — values miscited from a table the model was shown — and detect it with a trained critic; ProveML detects it with a lookup. Fact verification against evidence has a canonical benchmark lineage — FEVER (Thorne et al., 2018), TabFact (Chen et al., 2020), FEVEROUS (Aly et al., 2021) — in which a trained model judges support; ProveML sits outside it by making the judgment a lookup. Probabilistic faithfulness checkers, academic (SummaC, Laban et al., 2022; AlignScore, Zha et al., 2023; MiniCheck, Tang et al., 2024; HallDetect, Oukelmoun et al., 2026) and industrial, score responses after generation; Evergreen (Lee et al., 2026) verifies aggregate claims over relational data as queries, which ProveML can only bind once the aggregate is materialised as a fact; structured inline citation (Yeginbergen et al., 2026) and decode-time grammars whose reference slots admit only declared names (Zhang et al., 2026) are the constrained-generation route to the same end, and the natural next step for ProveML (Section 6). To our knowledge, no existing framework combines inline claim markup, deterministic mismatch detection against structured data, and composable threshold inference in a single system.[4] Table 5 places the neighbours; the technical

[3] Vera review protocol 1.0, `https://github.com/abovebeyond-ai/vera/blob/main/docs/review-protocol.md`: bind, flag, resolve, read, judge, with a written rubric for the resolve step and a per-round measure that ends the loop at zero.

[4] We screened the titles and abstracts of the 4,617 papers in the ACL 2026 main, short, findings and industry volumes against concept patterns for claim-level attribution, structured-data faithfulness, symbolic or deterministic verification, provenance and markup schemes, then read the resulting candidates by hand. The sweep was not preserved as a runnable artifact, so this is the result of a search rather than an exhaustive negative.

report discusses each in full.

Approach	Category	Determ.	Structured	Inline	Inference
FActScore, SAFE, FacTool, HallDetect	Fact-checking	–	–	–	–
Guardrail and grounding scorers	Guardrails	–	–	–	–
Schema validators, decode-time grammars	Schema	✓	–	–	–
ClaimDB, Evergreen, VeriFin	Data-grounded	Partial	✓	–	–
FullCite	Inline citation	–	–	✓	–
iXBRL*	Inline tagging	✓	✓	✓	–
SymGen	Inline tagging	✓	✓	✓	–
PCN	Inline tagging	✓	✓	✓	–
Claim-locked reporting	Inline tagging	✓	✓	✓	Partial
FinGround	Hybrid post-hoc	Partial	✓	–	–
ProveML	**Inline tagging**	✓	✓	✓	✓

Table 5: Comparison across representative verification approaches. Determ. = deterministic. Structured = against structured data. Inline = claim-level markup in text. Inference = composable threshold checks within natural language text. The guardrails row reflects the predominant model-based grounding mode, in which a trained model scores a response against its source and a threshold decides the outcome (Amazon Bedrock Guardrails contextual grounding check (Amazon Web Services, 2026b), Azure AI Content Safety groundedness detection (Microsoft, 2026), Vectara's HHEM (Vectara, 2026) and similar); Amazon Bedrock's separate Automated Reasoning checks add formal logic checks against a declared policy and validate only what the policy's variables capture, not author-marked claims in the text (Amazon Web Services, 2026a). Claim-locked reporting ties language strength to statistical thresholds but does not compose them. FinGround recomputes arithmetic claims deterministically but decomposes and types them with a model, hence Partial; the data-grounded row is Partial for the same reason, since Evergreen compiles claims to queries on an LLM query engine and VeriFin grounds and derives its operands with a model before the Z3 check. *The XBRL formula family provides document-level computation and assertion checks over the facts in a report (XBRL International, 2009a,b); the Inline XBRL specification is scoped to syntax and to mapping it into an XBRL instance, and defines no claim-scoped inference (XBRL International, 2013).

6 Discussion and Limitations

The deeper value is not the correction loop but the boundary itself. With ProveML markup, every marked claim carries a verification status and an addressable path, so a reader — or an auditor months later — can ask of any marked number where it came from and get an answer that does not depend on a model. What changes is the unit of accountability: from "was this report right" to "was this claim right, and against which record". Because entity references are structural and display text is separate from the identifier the verifier resolves, a deployment can hand the model pseudonymous identifiers and substitute names at display time — a property the syntax makes available, not one we evaluated.

- **Consistency, not truth.** ProveML verifies that claims match the fact store, not that the fact store is correct. If the source data is wrong, verified claims will be wrong. The review layer described under Deployment gives that boundary a discipline — evidence per store value, a human sign-off saved under a hash of exactly what it judged, expiring when the evidence changes — but does not remove it: a fair reading is a judgement, not a proof.
- **The right name, not necessarily the right record.** An entity verifies when the name the reader sees equals the name stored at the id the model chose; nothing checks that the model chose the right id. If two records of one type share a name, a wrong id renders

identically to the right one and only the audit path tells them apart. The verifier therefore measures whether a rendered subject is unique in the store (`subjectUnique`: true, false with the other paths, or unknown when the source cannot be enumerated), `doctor` warns on duplicate names, strict mode makes an ambiguous subject a finding, and the render marks it. Where subjects are identifiers by construction, as on a ledger, the gap closes; where they are personal names, it does not, and the marker is what remains.

- **No unit conversion.** Units are supported as nullable metadata with exact string matching. Semantic unit equivalence (e.g., mg/dL vs mmol/L) and standardized vocabularies (UCUM for units of measure (Schadow and McDonald, 2024), ISO 4217 for currency codes (SIX Financial Information, 2026)) are deferred to future work.
- **Evaluation scope.** The evaluation covers two domains (generated education, real SEC finance) across 38 prompts, 3 models, and 3 repeats, with an oracle retriever and one correction pass. Larger schemas, more domains, and retrieval under realistic conditions are not covered. The explicit-record fact form was introduced after the first study and measured in the second (Finding 3); whether a model follows the rule on every run is not guaranteed, as one Opus 5 run shows.
- **Judgments: one domain, one registry.** The judgment study covers one store, one registry of fourteen names and twenty questions, and its baseline is the facts-only prompt rather than free prose. How the construct behaves with a large registry, with judgments over derived values, or when the registry itself is contested, is not measured.
- **Canonical claims.** The model is not allowed to round: a claim says `391035000000`, never "$391 billion". The reader can still see the rounded form, because the store may declare a display rule per field (`_display`) that the renderer applies to verified values; but a value the model itself rounded is a mismatch, and prose that quotes the rounded figure outside a claim is unverified prose. The rule is on the store's side of the boundary on purpose: a tolerance in the verifier would make "about 18" verify against 18.5 and turn the guarantee into an approximation.
- **Prose outside markup.** ProveML verifies what is inside markup constructs. Text outside the markup is not checked. The verifier now measures the numeric part of this gap as coverage (90–96% in Section 3) and can refuse a number outside any claim, but a qualitative sentence with no number in it remains invisible to it. And a set of verified claims does not verify the sentence around them: Chan et al. (2026) argue that formally sound conclusions invite unsupported inferences by the reader, and nothing in ProveML's verdict speaks to what the prose between two verified claims implies.
- **Malformed input.** The tokenizer silently skips constructs with unmatched brackets or braces. Robustness to diverse malformed markup deserves further stress-testing.
- **Adversarial generation.** ProveML is designed against models that make mistakes, not against a model trying to mislead. A generator that wanted to could choose a threshold that holds and wrap misleading words around it, mark only the claims that verify and leave the damaging ones as prose, or select a true fact that misrepresents the record it came from. Each of these survives verification, the second wholly only outside strict mode or when the unmarked prose carries no number, because the verifier reads paths and values, not intent. We neither measure nor defend against this.
- **Group claims.** An inference takes its implicit context from a single entity. A claim like "5TW, 4B, and 6WE all have low coverage" verifies as one inference only if each atom names its record explicitly (`IS_LOW_COVERAGE(offering:a.evalRate) AND ...`); otherwise it must be decomposed into one inference per entity.

Future work follows from the limits: derived-value claims, standardized unit vocabularies, an abstention metric for unsupported queries, and constraining decoding to well-formed ProveML with only existing paths — grammar-constrained generation (PICARD, Scholak et al., 2021; Synchromesh, Poesia et al., 2022) makes that an application of known technique rather than a

research program, and it would remove much of what we measure as addressability error.

7 Conclusion

ProveML places AI-generated claims inside a verifiable boundary. Three inline constructs link every marked claim to an addressable path in a data source. Verification is deterministic, with no model in the loop. An optional threshold registry extends this to qualitative judgments through composable primitives.

The evaluation (Section 3) says that the mechanism is within reach of current models: three frontier models produce the markup on every query, cover 90–96% of their numbers with claims, and verify 92–100% of those claims after one correction. What remained was a property of the language more than of the models; the specification changed in response, and with the change in the prompt the residue went where the rule was followed. The earlier study of small local models (technical report) says the complement: below the frontier, whether a model produces verifiable markup at all depends on the shape of the request more than on the size of the model.

What LLMs provide is open-ended generation. What ProveML adds is a controllable, auditable boundary around that generation. For humans, the intended benefit is visual triage: verified claims carry a status, and attention shifts to what is unverified. We have not measured whether this reduces human verification effort, and the evidence from adjacent systems is mixed: Torroba Hennigen et al. (2024) report their user study reduced average verification time by 20%, while Mehrotra et al. (2026) report a 21-participant study in which verification and correction effort did not differ significantly from their baseline, even though their system reduced hallucination. Which of the two ProveML's rendering resembles in practice is an open empirical question. For agent loops the benefit is more direct and does not depend on human factors: specific error feedback ("`%[glucose]{143 mg/dL} in patient:308: should be 142 mg/dL`") enables targeted self-correction, which is what the verify-fix loop measures in Section 3. The combination is what neither open-ended generation nor static verification offers alone: open-endedness with determinism.

Declaration of Generative AI Use

Generative AI tools (Claude, GPT) were used during manuscript preparation for drafting, revision, literature search, and code review. All AI-generated content was reviewed, verified, and edited by the author. No generative AI was used to create or alter images or artwork. The author takes full responsibility for the content of the published article.

References

R. Aly, Z. Guo, M. Schlichtkrull, J. Thorne, A. Vlachos, C. Christodoulopoulos, O. Cocarascu, and A. Mittal. FEVEROUS: Fact extraction and VERification over unstructured and structured information. In *Proc. NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2106.05707.

Amazon Web Services. What are automated reasoning checks in amazon bedrock guardrails? Amazon Bedrock User Guide, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html, 2026a. Accessed 10 September 2026.

Amazon Web Services. Use contextual grounding check to filter hallucinations in responses. Amazon Bedrock User Guide, https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-contextual-grounding-check.html, 2026b. Accessed 10 September 2026.

Tianyu Cao, Natraj Raman, Danial Dervovic, and Chenhao Tan. Characterizing multimodal long-form summarization: A case study on financial reports. arXiv:2404.06162, 2024. URL https://arxiv.org/abs/2404.06162.

Jason Chan, Robert Gaizauskas, and Zhixue Zhao. Position: Logical soundness is not a reliable criterion for neurosymbolic fact-checking with LLMs. arXiv:2604.04177, 2026.

W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang. TabFact: A large-scale dataset for table-based fact verification. In *Proc. ICLR*, 2020. arXiv:1909.02164.

Elizabeth Clark, Tal August, Sofia Serrano, Nikita Haduong, Suchin Gururangan, and Noah A. Smith. All that's 'human' is not gold: Evaluating human evaluation of generated text. In *Proc. ACL-IJCNLP (Volume 1: Long Papers)*, pages 7282–7296, 2021. doi: 10.18653/v1/2021.acl-long.565.

Valentin Dinu and Prakash Nadkarni. Guidelines for the effective use of entity-attribute-value modeling for biomedical databases. *International Journal of Medical Informatics*, 76(11–12): 769–779, 2007. doi: 10.1016/j.ijmedinf.2006.09.023.

European AI Office. Code of practice on transparency of AI-generated content. June 2026, https://digital-strategy.ec.europa.eu/en/faqs/signing-code-practice-transparency-ai-generated-content, 2026.

European Commission. Guidelines on the implementation of the transparency obligations for certain AI systems under Article 50 of the AI Act. Adopted 20 July 2026, https://digital-strategy.ec.europa.eu/en/library/guidelines-transparency-obligations-providers-and-deployers-ai-systems, 2026.

European Parliament and Council of the European Union. Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence. Artificial Intelligence Act. Official Journal of the European Union, https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng, 2024.

European Parliament and Council of the European Union. Regulation (EU) 2026/1744 of the European Parliament and of the Council of 8 July 2026 amending regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as regards the simplification of the implementation of harmonised rules on artificial intelligence (Digital Omnibus on AI). Official Journal of the European Union, https://eur-lex.europa.eu/eli/reg/2026/1744/oj/eng, 2026.

Xiao Fan, Jingyuan Li, Hongbin Guo, Yubo Han, and Yi Zhang. Provenance before prose: Claim-locked reporting. arXiv:2608.25336, 2026.

T. Gao, H. Yen, J. Yu, and D. Chen. Enabling large language models to generate text with citations. In *Proc. EMNLP*, pages 6465–6488, 2023. doi: 10.18653/v1/2023.emnlp-main.398.

Bethel Hall, Sachi Shome, and William Eiers. VeriFin: A neurosymbolic framework for verifying LLM-generated financial claims. arXiv:2608.10213, 2026.

Maurice Jakesch, Jeffrey T. Hancock, and Mor Naaman. Human heuristics for AI-generated language are flawed. *Proceedings of the National Academy of Sciences*, 120(11):e2208839120, 2023. doi: 10.1073/pnas.2208839120.

Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023. doi: 10.1145/3571730.

Cameron R. Jones and Benjamin K. Bergen. Large language models pass a standard three-party Turing test. *Proceedings of the National Academy of Sciences*, 123(21):e2524472123, 2026. doi: 10.1073/pnas.2524472123.

A. T. Kalai and S. S. Vempala. Calibrated language models must hallucinate. In *Proc. STOC*, 2024. arXiv:2311.14648.

A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang. Why language models hallucinate. OpenAI, arXiv:2509.04664, 2025.

P. Laban, T. Schnabel, P. N. Bennett, and M. A. Hearst. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177, 2022. doi: 10.1162/tacl_a_00453.

Jiwon Lee, Seokhyun Han, Rathijit Sen, Jinsoo Yeom, Ugur Cetintemel, and Anindya Datta. Evergreen: Efficient claim verification for semantic aggregates. arXiv:2604.26180, 2026.

Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7001–7025, 2023. doi: 10.18653/v1/2023.findings-emnlp.467.

Patty Liu, Dominik Stammbach, and Peter Henderson. Who checks the citations? benchmarking legal hallucination detection. arXiv:2606.21155, 2026.

John MacFarlane. CommonMark spec. Version 0.31.2, https://spec.commonmark.org/0.31.2/, 2024.

V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho. Hallucination-free? assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 22(2):216–242, 2025. doi: 10.1111/jels.12413.

N. Mehrotra, A. Kumar, S. Gulwani, A. Radhakrishna, and A. Tiwari. TEN: Table explicitization, neurosymbolically. In *Proc. ACL Industry Track*, 2026. doi: 10.18653/v1/2026.acl-industry.138.

Microsoft. Groundedness detection in azure ai content safety - azure ai services. Azure AI Content Safety documentation, https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/groundedness, 2026. Accessed 10 September 2026.

Prakash M. Nadkarni, Luis Marenco, Roland Chen, Emmanouil Skoufos, Gordon Shepherd, and Perry Miller. Organization of heterogeneous scientific data using the EAV/CR representation. *Journal of the American Medical Informatics Association*, 6(6):478–493, 1999. doi: 10.1136/jamia.1999.0060478.

H. Onweller, E. Lumer, A. Huber, P. Ramchandani, V. K. Subbiah, and C. Feld. Cited but not verified: Parsing and evaluating source attribution in LLM deep research agents. arXiv:2605.06635, 2026.

Achir Oukelmoun, Nasredine Semmar, and Gaël De Chalendar. Decomposed entailment for factuality checking and hallucination detection. arXiv:2608.05823, 2026.

Yesim Ozarda. Reference intervals: current status, recent developments and future considerations. *Biochemia Medica*, 26(1):5–16, 2016. doi: 10.11613/BM.2016.001.

G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *Proc. ICLR*, 2022. arXiv:2201.11227.

Prometheus Authors. Alerting rules. Prometheus documentation, https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/, 2026. Undated; accessed 10 September 2026.

Delip Rao, Eric Wong, and Chris Callison-Burch. Detecting and correcting reference hallucinations in commercial LLMs and deep research agents. arXiv:2604.03173, 2026.

H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023. doi: 10.1162/coli_a_00486.

Jenna Russell, Marzena Karpinska, and Mohit Iyyer. People who frequently use ChatGPT for writing tasks are accurate and robust detectors of AI-generated text. In *Proc. ACL (Volume 1: Long Papers)*, pages 5342–5373, 2025. doi: 10.18653/v1/2025.acl-long.267.

Gunther Schadow and Clement J. McDonald. The unified code for units of measure. Version 2.2, Regenstrief Institute and the UCUM Organization, https://ucum.org/ucum, 2024.

T. Scholak, N. Schucher, and D. Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proc. EMNLP*, pages 9895–9901, 2021. doi: 10.18653/v1/2021.emnlp-main.779.

SIX Financial Information. ISO 4217 - currency codes. Maintenance Agency for ISO 4217, https://www.six-group.com/en/products-services/financial-information/data-standards.html, 2026.

Aivin V. Solatorio. Proof-carrying numbers (PCN): A protocol for trustworthy numeric answers from LLMs via claim verification. arXiv:2509.06902, 2025.

Stanford HAI. The 2026 AI index report. Stanford Institute for Human-Centered Artificial Intelligence, Economy chapter, https://hai.stanford.edu/ai-index/2026-ai-index-report/economy, 2026.

Mingjie Sun, Yida Yin, Zhiqiu Xu, J. Zico Kolter, and Zhuang Liu. Idiosyncrasies in large language models. In *Proc. ICML*, 2025. URL https://arxiv.org/abs/2502.12150. arXiv:2502.12150.

L. Tang, P. Laban, and G. Durrett. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proc. EMNLP*, 2024. doi: 10.18653/v1/2024.emnlp-main.499.

J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proc. NAACL-HLT*, pages 809–819, 2018. doi: 10.18653/v1/N18-1074.

L. Torroba Hennigen, S. Z. Shen, A. Nrusimha, B. Gapp, D. Sontag, and Y. Kim. Towards verifiable text generation with symbolic references. In *Proc. COLM*, 2024. arXiv:2311.09188.

Vectara. HHEM-2.1-open. Hugging Face, https://huggingface.co/vectara/hallucination_evaluation_model, 2026. Accessed 10 September 2026.

W3C. RDFa 1.1 primer — third edition. W3C Working Group Note, https://www.w3.org/TR/rdfa-primer/, 2015.

Aileen P. Wright, Carolynn K. Nall, Jacob J. H. Franklin, Sara N. Horst, Yaa A. Kumah-Crystal, Adam T. Wright, and Dara E. Mize. Enterprise-wide simultaneous deployment of ambient scribe technology: lessons learned from an academic health system. *Journal of the American Medical Informatics Association*, 33(2), 2026. doi: 10.1093/jamia/ocaf186.

Austin Wright, Henry Andrews, and Ben Hutton. JSON schema validation: A vocabulary for structural validation of JSON. Internet-Draft draft-bhutton-json-schema-validation-01, https://json-schema.org/draft/2020-12/json-schema-validation, 2022.

XBRL International. Validation 1.0. Recommendation 22 June 2009, https://www.xbrl.org/specification/validation/rec-2009-06-22/validation-rec-2009-06-22.html, 2009a.

XBRL International. Value assertions 1.0. Recommendation 22 June 2009, https://www.xbrl.org/specification/valueassertions/rec-2009-06-22/valueassertions-rec-2009-06-22.html, 2009b.

XBRL International. Inline XBRL part 1: Specification 1.1. https://www.xbrl.org/specification/inlinexbrl-part1/rec-2013-11-18/inlinexbrl-part1-rec-2013-11-18.html, 2013.

Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *Proc. ICLR*, 2024. URL https://arxiv.org/abs/2306.13063. arXiv:2306.13063.

Yuqing Yang, Qi Zhu, Zhen Han, Boran Han, Zhengyuan Shen, Shuai Wang, Vassilis N. Ioannidis, and Huzefa Rangwala. When LLMs read tables carelessly: Measuring and reducing data referencing errors. arXiv:2606.32029, 2026.

Anar Yeginbergen, Amelie Wührl, Anna Rogers, and Rodrigo Agerri. Explicit evidence grounding via structured inline citation generation. arXiv:2606.07130, 2026.

Y. Zha, Y. Yang, R. Li, and Z. Hu. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proc. ACL*, pages 11328–11348, 2023. doi: 10.18653/v1/2023.acl-long.634.

Shuoming Zhang, Ruiyuan Xu, Haofeng Li, Qiuchu Yu, Yangyu Zhang, Chunwei Xia, Xiaobing Feng, Chenxi Wang, Huimin Cui, and Jiacheng Zhao. Decode-time grammars: Constrained LLM generation over a refinement order of grammar fragments. arXiv:2607.18357, 2026.

Kaitlyn Zhou, Jena D. Hwang, Xiang Ren, and Maarten Sap. Relying on the unreliable: The impact of language models' reluctance to express uncertainty. In *Proc. ACL (Volume 1: Long Papers)*, pages 3623–3643, 2024. doi: 10.18653/v1/2024.acl-long.198.

A Error Detection Rate

Setup. The critical question for ProveML is not whether the LLM generates correct output, but whether ProveML *detects* incorrect output. To test this, we injected 20 deliberate errors into otherwise valid ProveML text and measured whether the verifier caught each one.

Results. The 20 errors span six categories: wrong values (5), wrong entity (3), no context (2), wrong threshold (4), cross-entity swaps (2), and subtle errors including trailing spaces and swapped entity order (4). ProveML detected all 20, yielding a **100% error detection rate**. This is a consequence of deterministic checking: a value that does not exactly match the store,

a name the registry does not hold, or a fact with no entity in force is flagged regardless of plausibility. The experiment validates that the implementation behaves as specified — it is a conformance test, not a detector benchmark — and it is one-sided: it measures detection of planted errors, not false positives on correct-but-reformatted values (a canonicalization difference such as `18.50` vs. `18.5` is flagged despite numeric equality; see Section 6). Robustness to malformed LLM markup is partially addressed by the correction-loop results in Section 3 but deserves further stress-testing.

B Formal Semantics

This appendix states what the verifier computes, as definitions, and what the paper claims for it, as propositions. The definitions and the proofs are mechanised in Lean 4 in the artifact repository (`formal/`, no dependencies beyond Lean); every proposition below names its theorem there. The mechanisation models the judgment, which verdict a construct receives, not the tokenizer that turns text into constructs; that link is covered by test vectors the model prints and the reference implementation must reproduce (`formal/check-vectors.mjs`, 11 of 11 agree).

Verdicts. A condition takes one of three values, **t**, **f** or **u** (unresolved). The connectives are Kleene's strong three-valued logic: $\neg\mathbf{u} = \mathbf{u}$; **f** wins a conjunction and **u** otherwise wins it; **t** wins a disjunction and **u** otherwise wins it. The information order puts **u** below both decided values: $a \sqsubseteq b$ iff $a = \mathbf{u}$ or $a = b$. All three connectives are monotone in it.

Store and registry. A store S is a partial function from paths `type:id.field` to values; a value is a number or a text. The surface value of a path, $\mathrm{surf}_S(q)$, is its value as text followed by the unit at q`._unit` when the store declares one. A registry R is a partial function from names to thresholds $(\mathit{field}, \mathit{op}, \mathit{unit}?)$; the operators are those of Table 1, with *between* read as $lo \le v < hi$ and *eq*, *neq*, *in* on the value's text. Numbers are a parameter with a decidable order; the model does not choose between integers and decimals.

An atom. A named threshold N in context c (the entity in force, or none), with an optional explicit path p, evaluates as follows. If $R(N)$ is undefined, **u**. Otherwise let $t = R(N)$ and let the path be p if p is given and addresses $t.\mathit{field}$, undefined if p is given and addresses another field, and $c.t.\mathit{field}$ if no p is given and c is defined; if the path is undefined, **u**. If $t.\mathit{op}$ is *is_null*, the verdict is **t** iff S has no value at the path. Otherwise, if S has no value there, **u**; if t declares a unit and the store's unit at the path is absent or different, **u**; if an ordering operator meets a text, **u**; else the operator's truth value. A label reference $@\ell$ is the verdict recorded under ℓ earlier in the document, **u** if none. Text the condition grammar rejects, a bare comparison included, is **u**.

A document. The tokenizer yields a sequence of constructs: an entity (path, rendered name, scoped or not), the close of a scope, a fact (a field of the entity in force, or an absolute path, and a value), or a judgment (label, condition). The verifier folds over the sequence with the entity in force, a stack of contexts, and the judgments so far by label. An entity becomes the entity in force and, if scoped, pushes the previous context, none included; a close pops it; a fact and a judgment leave the context as it is. Each construct receives one verdict: an entity is *verified* iff the store's name at its id equals the rendered name; a fact is *verified* iff its value equals the surface value at its path, *no context* if it is relative and no entity is in force; a judgment is *verified*, *failed* or *unverifiable* as its condition is **t**, **f** or **u**. Verification is a function of the document, the store and the registry, and of nothing else; determinism needs no proposition.

Propositions. Write $S \sqsubseteq S'$ when every value S holds, S' holds; likewise $R \sqsubseteq R'$ for registries.

1. **Verified means equal to the store.** If the verifier reports a fact at path q with value v as verified, then $\mathrm{surf}_S(q) = v$; if it reports an entity at p with name n as verified, then $S(p.\texttt{name})$ is defined and reads n. There is no tolerance anywhere. (`verify_fact_sound`, `verify_entity_sound`)
2. **A verified judgment rests on a registered bound that holds.** If an atom evaluates to **t**, its name is registered; and for every operator but *is_null*, the store holds a value at the bound path, the unit the threshold asks for is present and equal, and the operator holds of that value. (`evalAtom_tt_registered`, `evalAtom_tt_holds`)
3. **Nothing unresolved contributes to a verdict.** If $R \sqsubseteq R'$ then for every condition, its verdict under R is below its verdict under R' in the information order: registering more names can decide an unresolved judgment and can never flip a decided one. In particular a verified judgment stays verified and a failed one stays failed when the registry grows, and against the empty registry every judgment is unresolved. This is the formal content of "a model cannot make a claim checkable by naming a threshold the deployment never declared", and $\neg\mathbf{u} = \mathbf{u}$ is what makes it hold. (`evalCond_mono`, `evalCond_tt_stable`, `evalCond_ff_stable`, `evalCond_empty`)
4. **Closing a scope restores the context in force when it opened.** For a scoped entity around a body that opens and closes no scope of its own, the context after the close equals the context before the entity. (`scope_restores`)
5. **The store may grow too, with one exception.** If $S \sqsubseteq S'$ then an atom under any operator but *is_null* is below its verdict under S'. The exception is stated, not hidden: an *is_null* threshold evaluates to **t** wherever the store is silent, which includes a field the record's type never carries. That is the vacuity the judgment study met (Finding 4), and the specification should close it by distinguishing a missing field from a null value. (`evalAtom_store_mono`, `isNull_vacuous`)

What this establishes and what it does not. The propositions are about the model. The reference implementation is checked against it on the paper's examples, not proven to refine it; a verified implementation, the verifier written in the proof assistant and extracted, would close that gap and is the natural next step, and the tokenizer would then need modelling too. Beyond the mechanism nothing here is provable: that a verified claim is true is a property of the store, and that the text around it does not mislead is a property of the generating model. Both stay with the limitations in Section 6.

C ProveML Grammar (EBNF)

The following EBNF captures the ProveML constructs as implemented in the reference parser. It is intended as a starting point for formal treatment, not a normative specification; the authoritative definition is the reference implementation and test suite. Where the two diverge, the reference tokenizer is deliberately *more permissive* than this grammar — for example, it accepts identifiers beyond the character classes below and tolerates a missing space after the inference label colon — so the grammar describes the canonical surface form generators should produce, while the tokenizer accepts sloppier variants of it. One exception runs the other way: the tokenizer brace-counts entity display text, so an unbalanced open brace (admitted by the `display_text` production) causes the construct to be skipped.

```
(* ProveML constructs inside Markdown inline content *)
entity_ref_simple = "@[" , entity_path , "]{" , display_text , "}" ;
entity_ref_scoped = "@[" , entity_path , " " , DQUOTE ,
                    entity_name , DQUOTE , "]{" ,
                    scoped_content , "}" ;
entity_ref        = entity_ref_simple | entity_ref_scoped ;

fact_ref          = "%[" , field_name , "]{" , value_text , "}" ;
inference_ref     = "?[" , label , ": " , condition , "]{" ,
                    display_text , "}" ;

(* Paths and identifiers *)
entity_path       = identifier , ":" , entity_id ;
entity_id         = ( letter | digit ) ,
                    { letter | digit | "_" | "-" } ;

field_name        = field_segment , { "." , field_segment } ;
field_segment     = meta_field | path_field | identifier ;
meta_field        = "_" , identifier ;
path_field        = identifier , ":" , entity_id ;

label             = identifier ;
identifier        = letter , { letter | digit | "_" } ;

(* Text content *)
entity_name       = { char - '"' - ']' } ;
display_text      = { char - "}" } ;
scoped_content    = { char } ;   (* parsed recursively *)
value_text        = { char - "}" } ;

(* Condition grammar *)
condition         = or_expr ;
or_expr           = and_expr , { " OR " , and_expr } ;
and_expr          = unary_expr , { " AND " , unary_expr } ;
unary_expr        = [ "NOT " ] , primary ;
primary           = threshold_name
                  | threshold_name , "(" , path , ")"
                  | "@" , label ;

(* Terminals *)
threshold_name    = uppercase , { uppercase | digit | "_" } ;
path              = entity_path , "." , field_name ;
DQUOTE            = '"' ;
letter            = "a"-"z" | "A"-"Z" ;
uppercase         = "A"-"Z" ;
digit             = "0"-"9" ;
```